When a request arrives at the reverse proxy, it contains some information that is lost after the request is forwarded to your app. For example, the original request comes with the IP address of the client/browser connecting to your app; once the request is forwarded from the reverse proxy, the IP address is that of the *reverse proxy*, not the browser. Also, if the reverse proxy is used for SSL/TLS offloading (see chapter 28), then a request that was originally made using HTTPS may arrive at your app as an HTTP request.

The standard solution to these problems is for the reverse proxy to add more headers before forwarding requests to your app. For example, the `X-Forwarded-For` header identifies the original client's IP address, whereas the `X-Forwarded-Proto` header indicates the original scheme of the request (`http` or `https`).

For your app to behave correctly, it needs to look for these headers in incoming requests and modify the request as appropriate. A request to http://localhost with the `X-Forwarded-Proto` header set to `https` should be treated the same as if the request were to https://localhost.

You can use `ForwardedHeadersMiddleware` in your middleware pipeline to achieve this. This middleware overrides `Request.Scheme` and other properties on `HttpContext` to correspond to the forwarded headers. `WebApplicationBuilder` partially handles this for you; the middleware is automatically added to the pipeline in a disabled state. To enable it, set the environment variable `ASPNETCORE_FORWARDEDHEADERS_ENABLED=true`.

If you don't want to use the automatically added middleware for some reason, or if you're using the generic host (which you'll learn about in